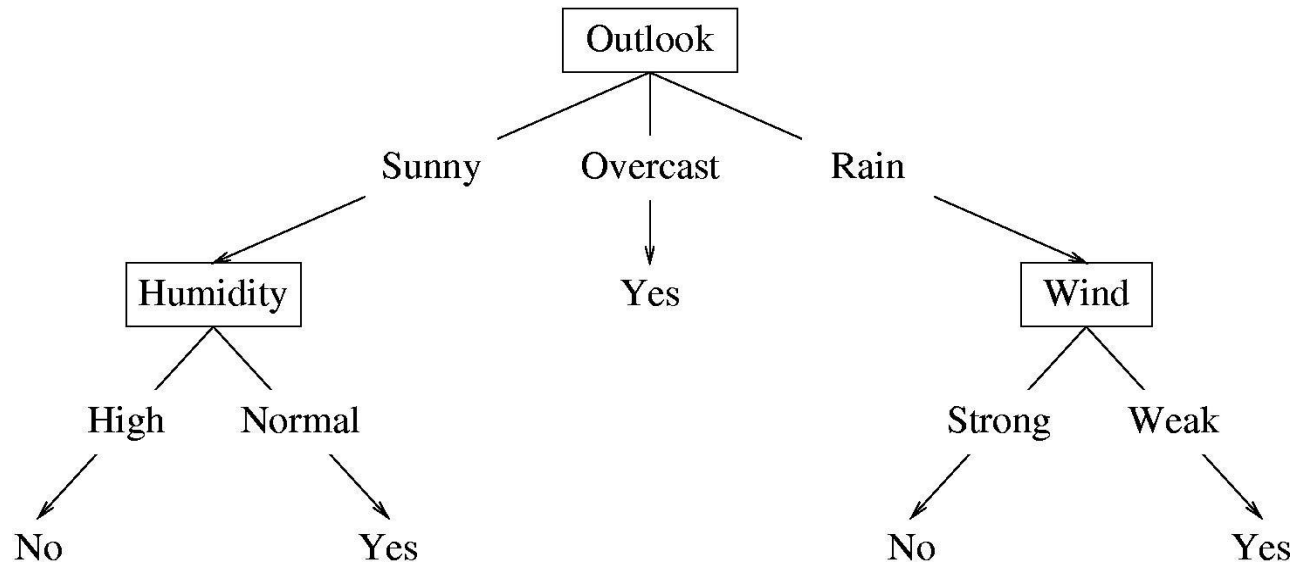




Decision Trees & Overfitting

Summary of Decision Trees (so far)



- Decision tree induction → choose the best attribute
 - Choose split via information gain
 - Build tree greedily, recursing on children of split
 - Stop when we achieve homogeny
 - i.e., when all instance in a child have the same class

Summary of Decision Trees (so far)

Information Gain: Mutual information of attribute A and the class variable of data set X

$$\begin{aligned} \text{InfoGain}(X, A) &= H(X) - H(X | A) \\ &= H(X) - \sum_{v \in \text{values}(A)} \underbrace{\frac{|\{x \in X \mid x_A = v\}|}{|X|}}_{\text{fraction of instances with value } v \text{ in attribute } A} \times \underbrace{H(\{x \in X \mid x_A = v\})}_{\text{entropy of those instances}} \end{aligned}$$

Entropy:

$$H(X) = - \sum_{c \in \text{Classes}} \underbrace{\frac{|\{x \in X \mid \text{class}(x) = c\}|}{|X|}}_{\text{fraction of instances of class } c} \log_2 \frac{|\{x \in X \mid \text{class}(x) = c\}|}{|X|}$$

Restaurant Example

Random: Patrons or Wait-time; **Least-values:** Patrons; **Most-values:** Type; **Max-gain:** ???

Type variable	Patrons variable		
	Empty	Some	Full
	French	Y	N
	Italian	Y	N
Thai	N	Y	N Y
Burger	N	Y	N Y

Computing information gain

$I(X) = ?$

$I(\text{Pat}, X) = ?$

$I(\text{Type}, X) = ?$

French		Y	N
Italian		Y	N
Thai	N	Y	N Y
Burger	N	Y	N Y
	Empty	Some	Full

Gain (Pat, X) = ?

Gain (Type, X) = ?

Computing information gain

$$\begin{aligned} I(X) &= \\ &- (.5 \log .5 + .5 \log .5) \\ &= .5 + .5 = 1 \end{aligned}$$

$$I(\text{Pat}, X) = ?$$

French		Y	N
Italian		Y	N
Thai	N	Y	N Y
Burger	N	Y	N Y
	Empty	Some	Full

$$I(\text{Type}, X) = ?$$

$$\text{Gain}(\text{Pat}, X) = ?$$

$$\text{Gain}(\text{Type}, X) = ?$$

Computing information gain

$$\begin{aligned} I(X) &= \\ &- (.5 \log .5 + .5 \log .5) \\ &= .5 + .5 = 1 \end{aligned}$$

$$\begin{aligned} I(\text{Pat}, X) &= \\ &2/12 (0) + 4/12 (0) + \\ &6/12 (- (4/6 \log 4/6 + \\ &\quad 2/6 \log 2/6)) \\ &= 1/2 (2/3 * .6 + \\ &\quad 1/3 * 1.6) \\ &= .47 \end{aligned}$$

$$I(\text{Type}, X) = ?$$

French		Y	N
Italian		Y	N
Thai	N	Y	N Y
Burger	N	Y	N Y
	Empty	Some	Full

$$\text{Gain}(\text{Pat}, X) = ?$$

$$\text{Gain}(\text{Type}, X) = ?$$

Computing information gain

$$\begin{aligned} I(X) &= \\ &- (.5 \log .5 + .5 \log .5) \\ &= .5 + .5 = 1 \end{aligned}$$

$$\begin{aligned} I(\text{Pat}, X) &= \\ &2/12 (0) + 4/12 (0) + \\ &6/12 (- (4/6 \log 4/6 + \\ &\quad 2/6 \log 2/6)) \\ &= 1/2 (2/3 * .6 + \\ &\quad 1/3 * 1.6) \\ &= .47 \end{aligned}$$

$$\begin{aligned} I(\text{Type}, X) &= \\ &2/12 (1) + 2/12 (1) + \\ &4/12 (1) + 4/12 (1) = 1 \end{aligned}$$

French	Y N		
Italian	Y N		
Thai	N	Y	N Y
Burger	N	Y	N Y
	Empty	Some	Full

Gain (Pat, X) = ?

Gain (Type, X) = ?

Computing information gain

$$\begin{aligned} I(X) &= \\ &- (.5 \log .5 + .5 \log .5) \\ &= .5 + .5 = 1 \end{aligned}$$

$$\begin{aligned} I(\text{Pat}, X) &= \\ &2/12 (0) + 4/12 (0) + \\ &6/12 (- (4/6 \log 4/6 + \\ &\quad 2/6 \log 2/6)) \\ &= 1/2 (2/3 * .6 + \\ &\quad 1/3 * 1.6) \\ &= .47 \end{aligned}$$

$$\begin{aligned} I(\text{Type}, X) &= \\ &2/12 (1) + 2/12 (1) + \\ &4/12 (1) + 4/12 (1) = 1 \end{aligned}$$

French		Y	N
Italian		Y	N
Thai	N	Y	N Y
Burger	N	Y	N Y
	Empty	Some	Full

$$\text{Gain}(\text{Pat}, X) = 1 - .47 = .53$$

$$\text{Gain}(\text{Type}, X) = 1 - 1 = 0$$

Attributes with Many Values

- Problem
 - If attribute has many values, InfoGain() will select it
 - e.g., imagine using `date = Jan_28_2011` as an attribute
- Alternative approach: use GainRatio() instead

$$\text{GainRatio}(X, A) = \frac{\text{InfoGain}(X, A)}{\text{SplitInformation}(X, A)}$$

$$\text{SplitInformation}(X, A) = - \sum_{v \in \text{values}(A)} \frac{|X_v|}{|X|} \log_2 \frac{|X_v|}{|X|}$$

where X_v is a subset of X for which A has value v

Computing Gain Ratio

Already computed:

- $I(X) = 1$
- $I(\text{Pat}, X) = 0.47$
- $I(\text{Type}, X) = 1$
- $\text{Gain}(\text{Pat}, X) = 0.53$
- $\text{Gain}(\text{Type}, X) = 0$

French		Y	N
Italian		Y	N
Thai	N	Y	N Y
Burger	N	Y	N Y
	Empty	Some	Full

$$\begin{aligned} \text{SplitInfo}(\text{Pat}, X) &= - (1/6 \log 1/6 + 1/3 \log 1/3 + 1/2 \log 1/2) \\ &= 1/6 * 2.6 + 1/3 * 1.6 + 1/2 * 1 = 1.47 \end{aligned}$$

$$\begin{aligned} \text{SplitInfo}(\text{Type}, X) &= 1/6 \log 1/6 + 1/6 \log 1/6 + 1/3 \log 1/3 + 1/3 \log 1/3 \\ &= 1/6 * 2.6 + 1/6 * 2.6 + 1/3 * 1.6 + 1/3 * 1.6 = 1.93 \end{aligned}$$

Computing Gain Ratio

Already computed:

- $I(X) = 1$
- $I(\text{Pat}, X) = 0.47$
- $I(\text{Type}, X) = 1$
- $\text{Gain}(\text{Pat}, X) = 0.53$
- $\text{Gain}(\text{Type}, X) = 0$

French		Y	N
Italian		Y	N
Thai	N	Y	N Y
Burger	N	Y	N Y
	Empty	Some	Full

$$\text{SplitInfo}(\text{Pat}, X) = - (1/6 \log 1/6 + 1/3 \log 1/3 + 1/2 \log 1/2)$$

$$= 1/6 * 2.6 + 1/3 * 1.6 + 1/2 * 1 = 1.47$$

$$\text{SplitInfo}(\text{Type}, X) = 1/6 \log 1/6 + 1/6 \log 1/6 + 1/3 \log 1/3 + 1/3 \log 1/3$$

$$= 1/6 * 2.6 + 1/6 * 2.6 + 1/3 * 1.6 + 1/3 * 1.6 = 1.93$$

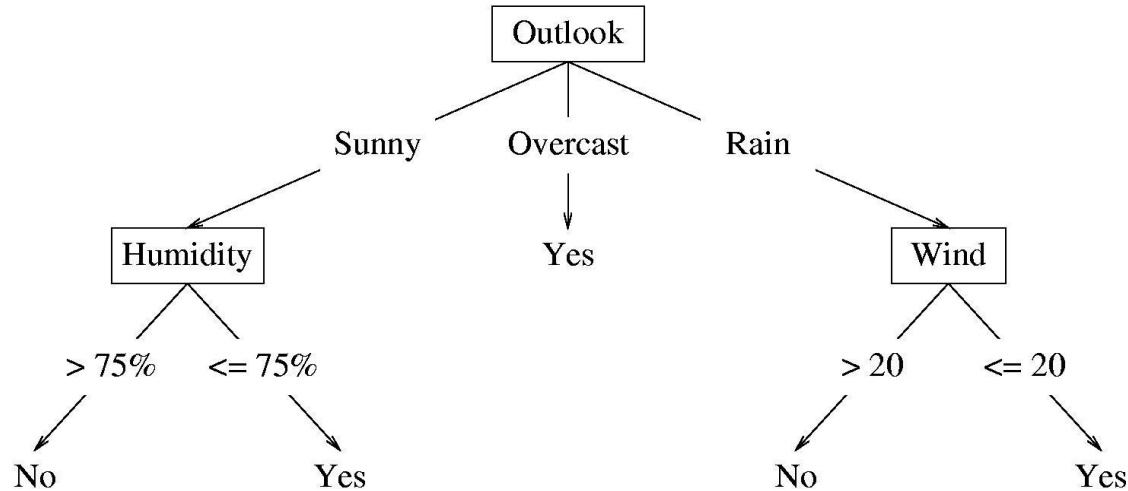
$$\text{GainRatio}(\text{Pat}, X) = \text{Gain}(\text{Pat}, X) / \text{SplitInfo}(\text{Pat}, X) = 0.53 / 1.47 = 0.36$$

$$\text{GainRatio}(\text{Type}, X) = \text{Gain}(\text{Type}, X) / \text{SplitInfo}(\text{Type}, X) = 0 / 1.93 = 0$$

Extensions of ID3

- Using gain ratios
- Real-valued data
- Noisy data and overfitting
- Generation of rules
- Setting parameters
- Cross-validation for experimental validation of performance
- C4.5 is an extension of ID3 that accounts for unavailable values, continuous attribute value ranges, pruning of decision trees, rule derivation, and so on

Real-Valued Features



- Change to binary splits by choosing a threshold
- One method:
 - Sort instances by value, identify adjacencies with different classes

Temperature:	40	48	60	72	80	90
PlayTennis:	No	No	Yes	Yes	Yes	No

candidate splits

- Choose among splits by InfoGain()

Unknown Attribute Values

What if some examples are missing values of A ?

Use training example anyway, sort through tree:

- If node n tests A , assign most common value of A among other examples sorted to node n
- Assign most common value of A among other examples with same class label
- Assign probability p_i to each possible value v_i of A .

Assign fraction p_i of example to each descendent of tree

Classify new examples in same fashion

Noisy Data

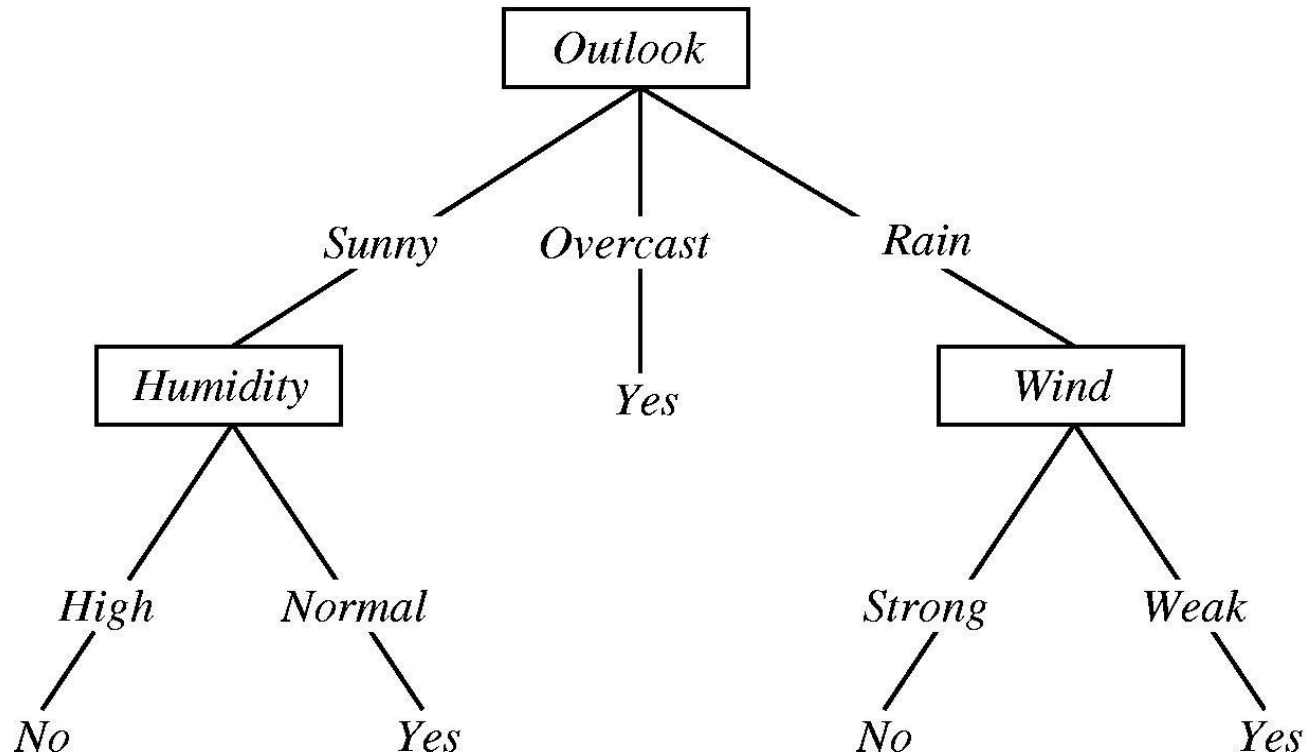
- Many kinds of “noise” can occur in the examples:
 - Two examples have same attribute/value pairs, but different classifications
 - Some values of attributes are incorrect because of errors in the data acquisition process or the preprocessing phase
 - The instance was labeled incorrectly (+ instead of -)
- Also, some attributes are irrelevant to the decision--making process
 - e.g., color of a die is irrelevant to its outcome

Overfitting

- Irrelevant attributes can result in *overfitting* the training example data
 - If hypothesis space has many dimensions (large number of attributes), we may find **meaningless regularity** in the data that is irrelevant to the true, important, distinguishing features
- If we have too little training data, even a reasonable hypothesis space will ‘overfit’

Overfitting in Decision Trees

Consider adding a noisy training example to the following tree:



What would be the effect of adding:

<outlook=sunny, temperature=hot, humidity=normal, wind=strong, playTennis=No> ?

Overfitting

Consider error of hypothesis h over

- training data: $error_{train}(h)$
- entire distribution $\mathcal{D}$ of data: $error_{\mathcal{D}}(h)$

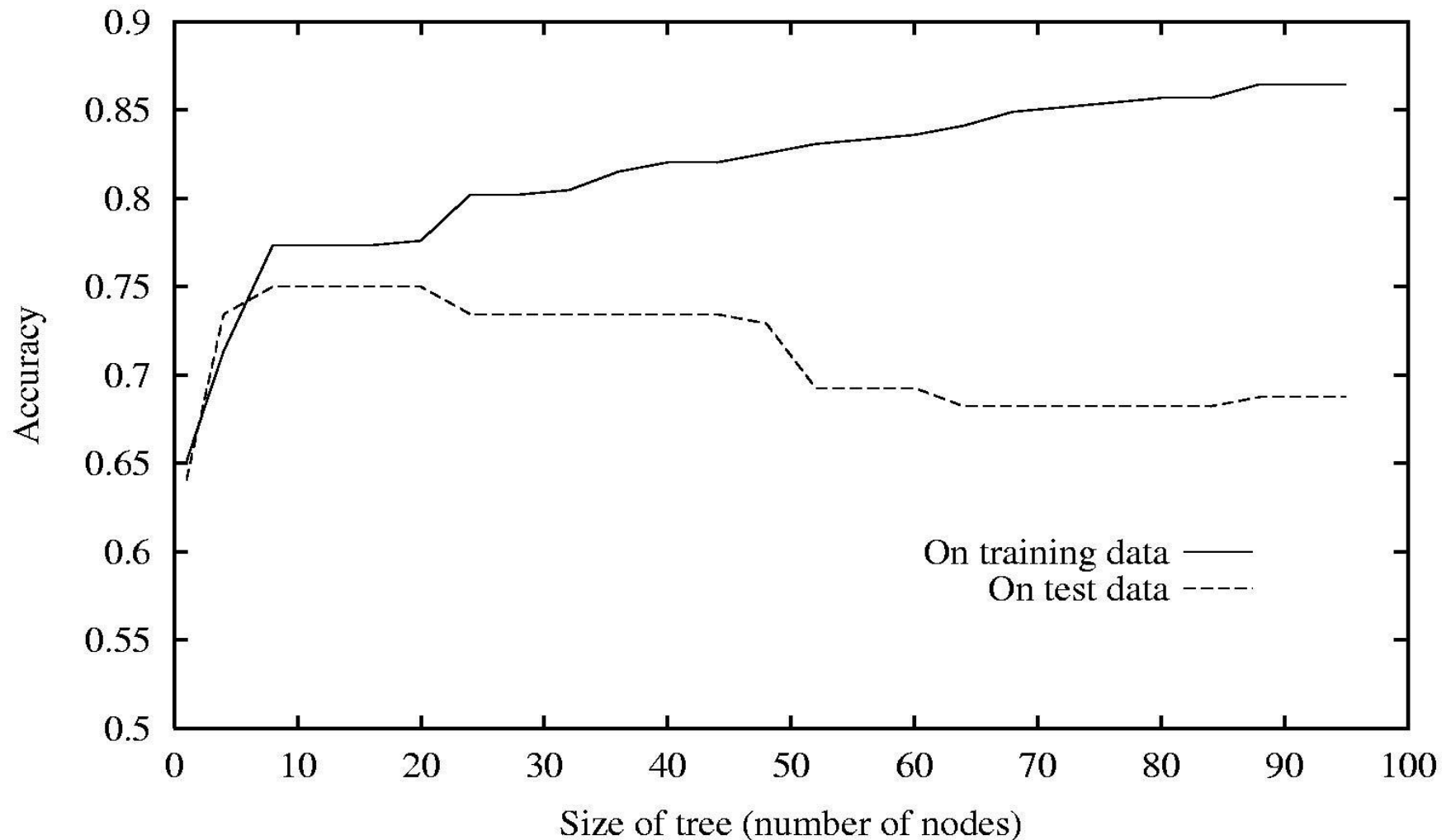
Hypothesis $h \in H$ **overfits** training data if there is an alternative hypothesis $h' \in H$ such that

$$error_{train}(h) < error_{train}(h')$$

and

$$error_{\mathcal{D}}(h) > error_{\mathcal{D}}(h')$$

Overfitting in Decision Tree Learning



Avoiding Overfitting

How can we avoid overfitting?

- Stop growing when data split is not statistically significant
- Acquire more training data
- Remove irrelevant attributes (manual process – not always possible)
- Grow full tree, then post-prune

How to select “best” tree:

- Measure performance over training data
- Measure performance over separate validation data set
- Add complexity penalty to performance measure

Reduced--Error Pruning

Split data into *training* and *validation* sets

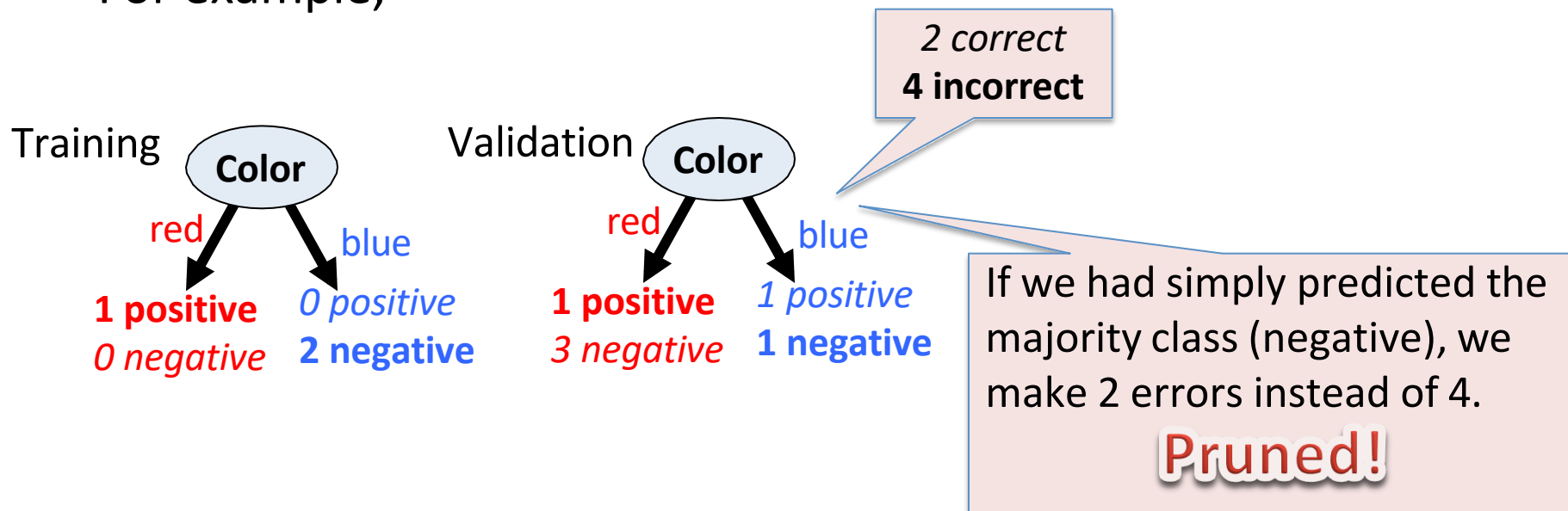
Grow tree based on *training set*

Do until further pruning is harmful:

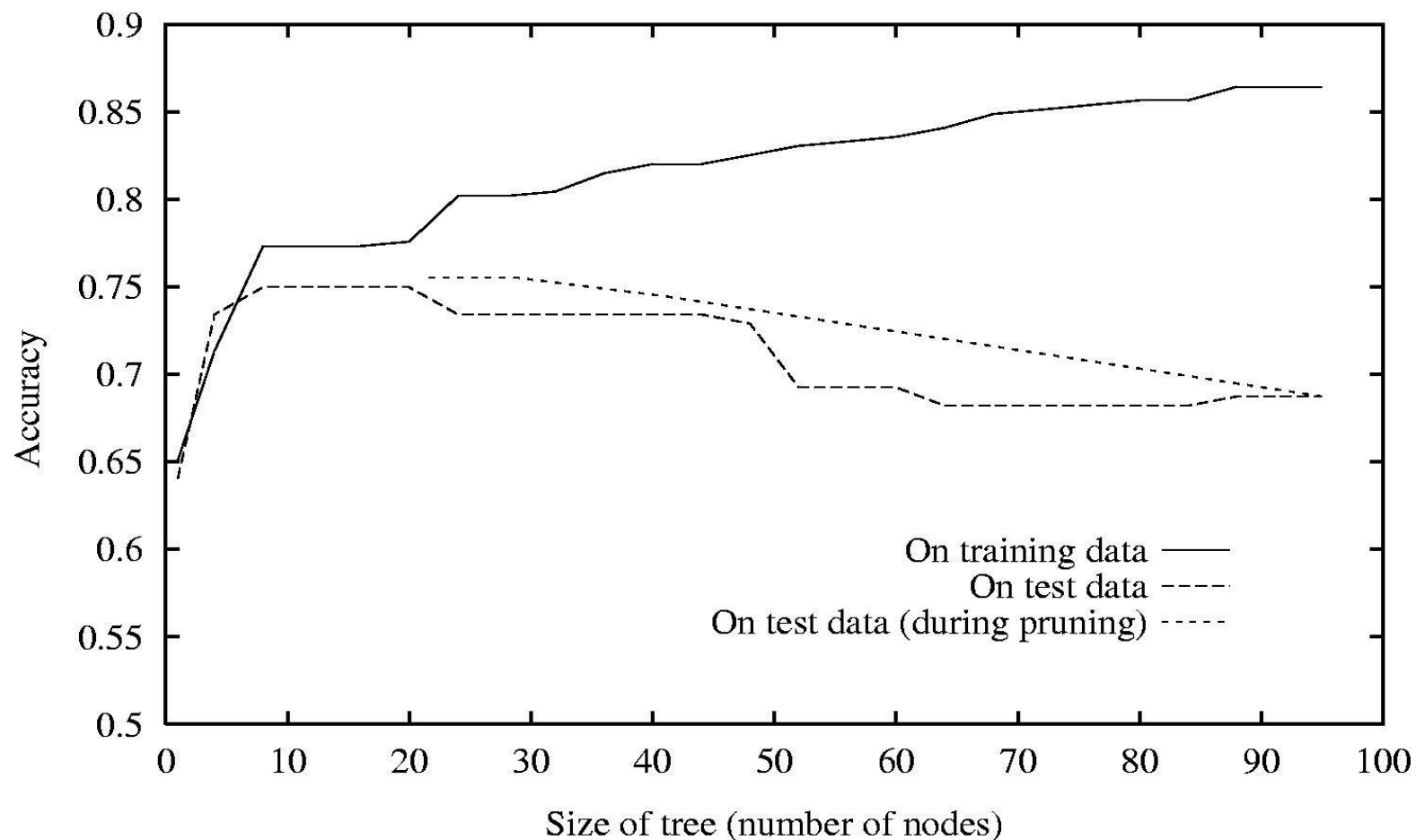
1. Evaluate impact on validation set of pruning each possible node (plus those below it)
2. Greedily remove the node that most improves *validation set* accuracy

Pruning Decision Trees

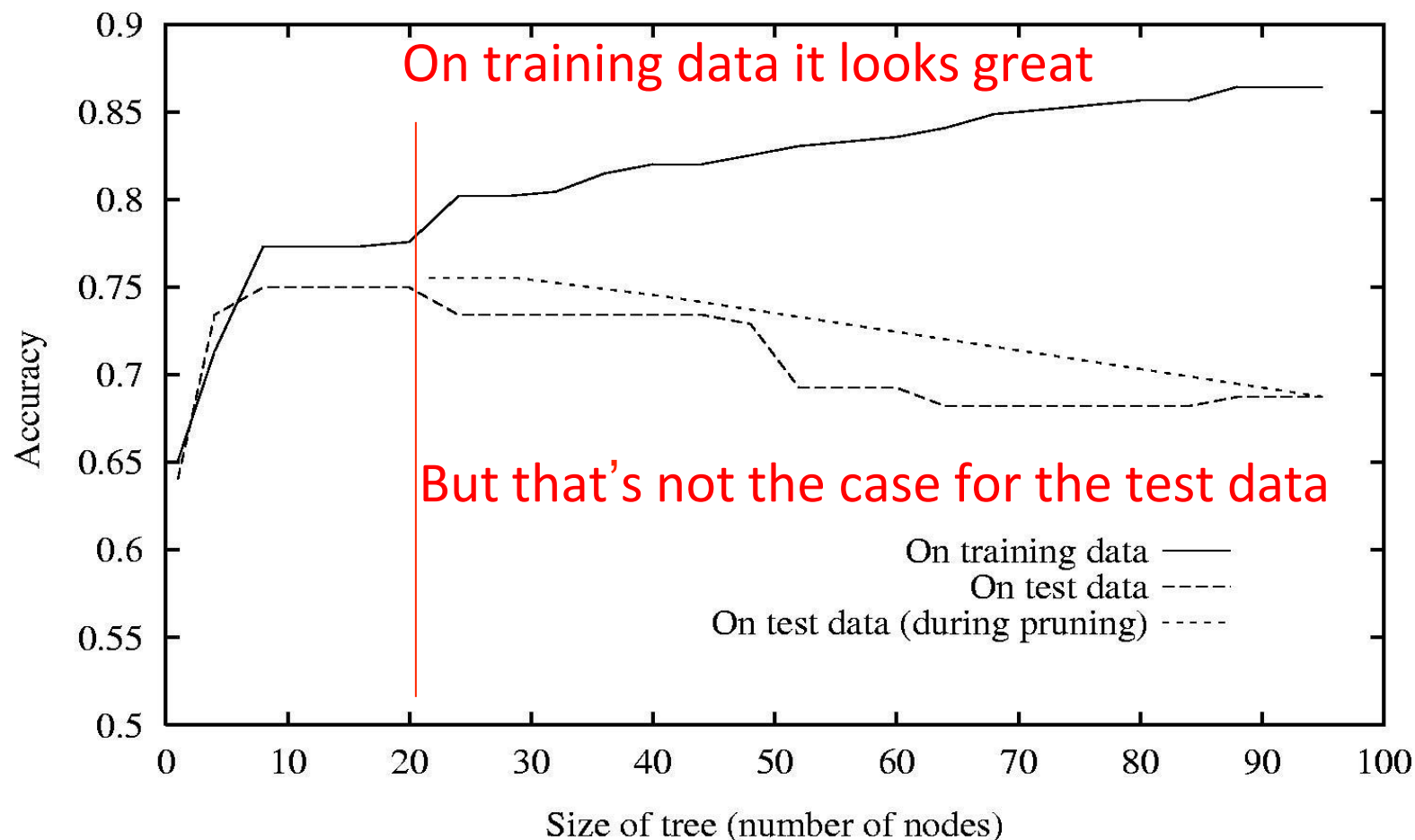
- Pruning of the decision tree is done by replacing a whole subtree by a leaf node.
- The replacement takes place if a decision rule establishes that the expected error rate in the subtree is greater than in the single leaf.
- For example,



Effect of Reduced--Error Pruning

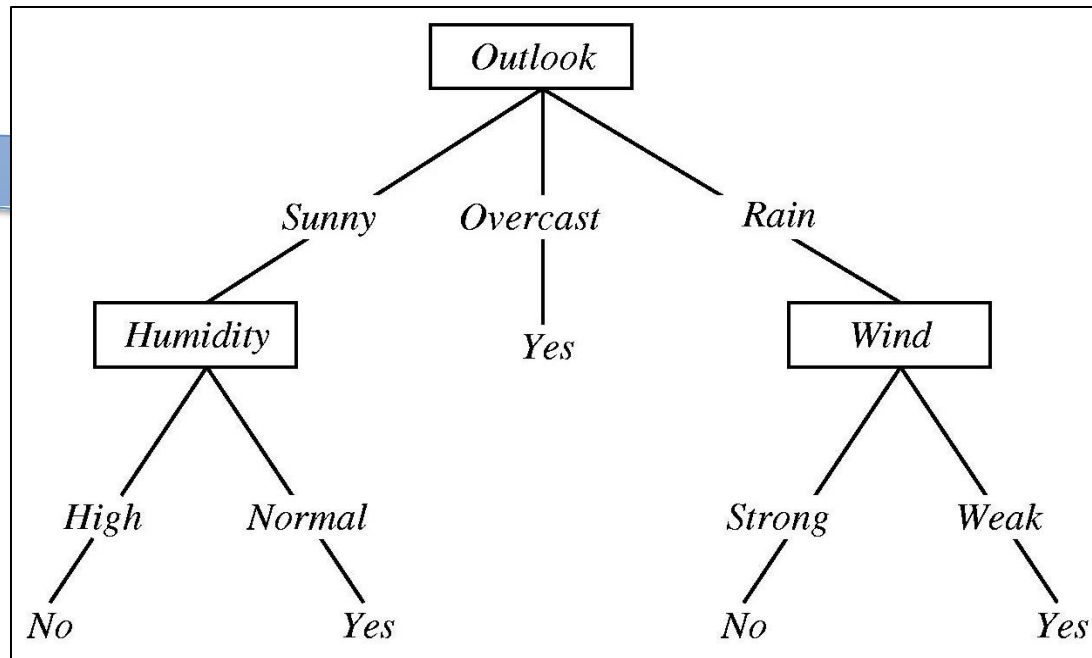


Effect of Reduced--Error Pruning



The tree is pruned back to the red line where it gives more accurate results on the test data

Converting a Tree to Rules



IF (*Outlook = Sunny*) AND (*Humidity = High*)
THEN *PlayTennis = No*

IF (*Outlook = Sunny*) AND (*Humidity = Normal*)
THEN *PlayTennis = Yes*

...

Converting Decision Trees to Rules

- It is easy to derive rules from a decision tree: write a rule for each path from the root to a leaf

(Outlook = Sunny) AND (Humidity = High) $\rightarrow$ PlayTennis = No

- To simplify the resulting rule set:
 - Let LHS be the left-hand side of a rule
 - LHS' obtained from LHS by eliminating some conditions
 - Replace LHS by LHS' in this rule if the subsets of the training set satisfying LHS and LHS' are equal
 - A rule may be eliminated by using meta-conditions such as “if no other rule applies”

Rule Post-Pruning

1. Convert tree to equivalent set of rules
2. Prune each rule independently of others
3. Sort final rules into desired sequence for use

Perhaps most frequently used method (e.g., C4.5)

Scaling Up

- ID3, C4.5, etc.: assumes that data fits in memory
(OK for up to hundreds of thousands of examples)
- SPRINT, SLIQ: multiple sequential scans of data
(OK for up to millions of examples)
- VFDT: at most one sequential scan
(OK for up to billions of examples)

Comparison of Learning Methods

Characteristic	Neural Nets	SVM	Trees	MARS	k-NN, Kernels
Natural handling of data of “mixed” type	▼	▼	▲	▲	▼
Handling of missing values	▼	▼	▲	▲	▲
Robustness to outliers in input space	▼	▼	▲	▼	▲
Insensitive to monotone transformations of inputs	▼	▼	▲	▼	▼
Computational scalability (large N)	▼	▼	▲	▲	▼
Ability to deal with irrelevant inputs	▼	▼	▲	▲	▼
Ability to extract linear combinations of features	▲	▲	▼	▼	◆
Interpretability	▼	▼	◆	▲	▼
Predictive power	▲	▲	▼	◆	▲

Summary: Decision Tree Learning

- Representation: decision trees
- Bias: prefer small decision trees
- Search algorithm: greedy
- Heuristic function: information gain or information content or others
- Overfitting / pruning

Summary: Decision Tree Learning

- Widely used in practice
- Strengths include
 - Fast and simple to implement
 - Can convert to rules
 - Handles noisy data
- Weaknesses include
 - Univariate splits/partitioning using only one attribute at a time —limits types of possible trees
 - Large decision trees may be hard to understand
 - Requires fixed-length feature vectors
 - Non-incremental (i.e., batch method)
 - Sacrifices predictive power

Class Activity: Build Your Own Decision Tree

Question 1

Calculate **Entropy** of the dataset

Outlook	Temperature	Humidity	Wind	PlayTennis
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Question 2

Compute **Information Gain** for these attributes:

- Outlook
- Humidity
- Wind

Question 3

Which attribute should be the root node?

Question 4

Draw the **first level of the decision tree**

Class Activity

- What attribute did you select?
- Why?

- Why do decision trees overfit?
- How does **pruning** help?
- Why is **Gain Ratio better than Information Gain** sometimes?